

24MCADMT201 ADVANCED DATABASE MANAGEMENT SYSTEMS

UNIT 4:

Data Storage and Querying:- RAID – File Organization – Organization of Records in Files – Indexing and Hashing: Basic concept, Ordered Indices, B+ tree Index Files: Structure of a B+-Tree (structure only, algorithms not needed) - B tree index files – Static Hashing – Dynamic Hashing – Query Processing: Overview - Selection Operation.

Data Storage and Querying

Database System is a software system, having an organized collection of structured information, stored in a computer system. It enables the user to create, maintain, define, and control access to the database.

The Database System is further divided into two components. They are as follows:

- Data Storage Manager
- Query Manager

Data Storage Manager

Data Storage Manager also known as “Database Control System”, is generally a program that provides an interface between the data/information stored and the queries received. It helps us to maintain the integrity and consistency of the database by applying the constraints.

Storage Manager is generally in charge of the interactions with the File Manager, where raw data is stored on the data with the help of the file system. It translates various DML statements into low-level commands.

- **Authorization and Integrity Manager:** The main purpose of the Authorization and Integrity Manager is to ensure the satisfaction of the integrity constraints and checks the authority of users to access information.
- **Transaction Manager:** The main purpose of Transaction Manager is to ensure that even after the system failures, the database should remain in a uniform state.
- **File Manager:** The main purpose of File Manager is to manage the allocation of space on the disk storage.
- **Buffer Manager:** The main purpose of Buffer Manager is to fetch the data from disk storage into the main memory. The buffer manager is a critical part of the database system since it enables the database to handle data sizes that are much larger than the size of the main memory.

Query Processor

Query Processor contains the following three components:

- **DDL Interpreter:** The DDL Interpreter is the Data Definition Language Interpreter which is used to build and modify the structure of your tables and other objects in the table.
- **DML Compiler:** DML Compiler is used for translating the DML statements into query language consists of low-level instructions that only query engines can understand. The DML compiler also performs query optimization; that is, it picks the lowest cost evaluation plan from among the alternatives.
- **Query Evaluation Engine:** It executes the low-level language instructions, generated by DML Compiler.

RAID (Redundant Array of Independent Disks)

RAID (Redundant Array of Independent Disks) in a Database Management System (DBMS) is a technology that combines multiple physical disk drives into a single logical unit for data storage. The main purpose of RAID is to improve data reliability, availability, and performance.

Simple example to understand the concept:

Let's understand how RAID (Redundant Array of Independent Disks) works through a relatable example:

Imagine you have a favourite book that you want to keep safe. Instead of giving the entire book to just one friend for safekeeping, you take a smart approach:

1. ***Splitting the Book:*** You divide the book into smaller pieces (like chapters or sections) and give each piece to a different friend.
2. ***Making Copies:*** For extra security, you might also create duplicate pieces and give them to multiple friends.

Now, if one friend misplaces their piece, you can still recreate the entire book using the pieces held by the others. This way, your book is safe even if someone loses their portion.

This is exactly how RAID works with hard drives! RAID splits your data across multiple drives

Different RAID Levels

- RAID-0 (Striping)
- RAID-1 (Mirroring)
- RAID-2 (Bit-Level Striping with Dedicated Parity)
- RAID-3 (Byte-Level Striping with Dedicated Parity)
- RAID-4 (Block-Level Striping with Dedicated Parity)
- RAID-5 (Block-Level Striping with Distributed Parity)
- RAID-6 (Block-Level Striping with two Parity Bits)

RAID-0 (Stripping)

RAID-0 improves system performance by splitting data into smaller “blocks” and spreading them across multiple disks. This process is called “striping.” It enhances data access speed by enabling parallel read/write operations but provides no redundancy or fault tolerance.

RAID 0

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

- A set of blocks distributed across the disks forms a “stripe.” For instance, “0, 1, 2, 3” is one stripe.
- Instead of placing just one block into a disk at a time, we can work with two (or more) blocks placed into a disk before moving on to the next one.

Evaluation

- **Reliability:** 0
There is no duplication of data. Hence, a block once lost cannot be recovered.
- **Capacity:** $N \times B$
The entire space is being used to store data. Since there is no duplication, N disks each having B blocks are fully utilized.

Advantages

- It is easy to implement.
- It utilizes the storage capacity in a better way.

Disadvantages

- A single drive loss can result in the complete failure of the system.
- It’s not a good choice for a critical system.

RAID-1 (Mirroring)

- RAID-1 enhances reliability by creating an identical copy (mirror) of each data block on separate disks. This ensures that even if one disk fails, the data remains accessible from its duplicate. While this configuration is highly reliable, it requires significant storage overhead.

RAID 1

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

Mirroring:

Each block of data is written to two (or more) disks.

For example:

- Block 0 is stored on Disk 0 and its duplicate on Disk 1.
- Block 1 is stored on Disk 2 and its duplicate on Disk 3.

Mirroring

Level

2:

In the figure, “mirroring level 2” indicates that there are two copies of each block, distributed across different disks.

RAID 0 was unable to tolerate any disk failure. But RAID 1 is capable of reliability.

Evaluation

Assume a RAID system with mirroring level 2.

- **Reliability:** 1 to $N/2$
1 disk failure can be handled for certain because blocks of that disk would have duplicates on some other disk. If we are lucky enough and disks 0 and 2 fail, then again this can be handled as the blocks of these disks have duplicates on disks 1 and 3. So, in the best case, $N/2$ disk failures can be handled.
- **Capacity:** $N*B/2$
Only half the space is being used to store data. The other half is just a mirror of the already stored data.

Advantages

- It covers complete redundancy.
- It can increase data security and speed.

Disadvantages

- It is highly expensive.
- Storage capacity is less.

RAID-2 (Bit-Level Striping with Dedicated Parity)

- RAID-2 is a specialized RAID level that uses bit-level striping combined with error correction using Hamming Code. In this configuration, data is distributed at the bit level across multiple drives, and a dedicated parity drive is used for error detection and correction. While it offers strong fault tolerance, its complexity and cost make it rarely used in practice.

Advantages

- In case of Error Correction, it uses hamming code.
- It Uses one designated drive to store parity.

Disadvantages

- It has a complex structure and high cost due to extra drive.
- It requires an extra drive for error detection.

RAID-3 (Byte-Level Striping with Dedicated Parity)

- RAID-3 enhances fault tolerance by employing byte-level striping across multiple drives and storing parity information on a dedicated parity drive. The dedicated parity drive allows for the reconstruction of lost data if a single drive fails. This configuration is suitable for workloads requiring high throughput for sequential data but is less efficient for random I/O operations.

RAID 3

Disk 0	Disk 1	Disk 2	Disk 3
15	16	17	P(15, 16, 17)
18	19	20	P(18, 19, 20)
21	22	23	P(21, 22, 23)
24	25	26	P(24, 25, 26)

- Here Disk 3 contains the Parity bits for Disk 0, Disk 1, and Disk 2. If data loss occurs, we can construct it with Disk 3.

Evaluation

Reliability: RAID-3 can tolerate the failure of one disk. The lost data can be reconstructed using the parity drive and the remaining data drives.

Capacity: Usable Capacity=(N-1)×B where N is the total number of drives, and B is the number of blocks per drive. The capacity of one drive is reserved for storing parity information.

Advantages

- Data can be transferred in bulk.
- Data can be accessed in parallel.

Disadvantages

- It requires an additional drive for parity.
- In the case of small-size files, it performs slowly.

RAID-4 (Block-Level Striping with Dedicated Parity)

- RAID-4 introduces block-level striping across multiple disks, combined with a dedicated parity disk to provide fault tolerance. Data is written in blocks, and separate disk stores parity information calculated using the XOR function. This setup allows for data recovery in case of a single disk failure, making RAID-4 more reliable than RAID-0 but less efficient in write-intensive scenarios due to reliance on a dedicated parity disk.

RAID 4

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

- In the figure, we can observe one column (disk) dedicated to parity.
- Parity is calculated using a simple XOR function. If the data bits are 0,0,0,1 the parity bit is $\text{XOR}(0,0,0,1) = 1$. If the data bits are 0,1,1,0 the parity bit is $\text{XOR}(0,1,1,0) = 0$. A simple approach is that an even number of ones results in parity 0, and an odd number of ones results in parity 1.

-

C1	C2	C3	C4	Parity
0	0	0	1	1
0	1	1	0	0

- Assume that in the above figure, C3 is lost due to some disk failure. Then, we can recompute the data bit stored in C3 by looking at the values of all the other columns and the parity bit. This allows us to recover lost data.

Evaluation

- Reliability: 1**
RAID-4 allows recovery of at most 1 disk failure (because of the way parity works). If more than one disk fails, there is no way to recover the data.
- Capacity: $(N-1)*B$**
One disk in the system is reserved for storing the parity. Hence, $(N-1)$ disks are made available for data storage, each disk having B blocks.

Advantages

- It helps in reconstructing the data if at most one data is lost.

Disadvantages

- It can't help reconstructing data when more than one is lost.

RAID-5 (Block-Level Striping with Distributed Parity)

- RAID-5 builds on RAID-4 by distributing parity information across all disks instead of storing it on a dedicated parity drive. This distributed parity significantly improves write performance, especially for random write operations, while maintaining fault tolerance for single disk failures. RAID-5 is one of the most commonly used RAID configurations due to its balance between reliability, performance, and storage efficiency.

RAID 5

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

How RAID-5 Works?

Block-Level Striping: Data is divided into blocks and striped across all drives in the array.

Distributed Parity: Parity bits, calculated using the XOR function, are distributed across all drives in a rotating pattern.

For example:

- Disk 0: Data (D1, D2), Parity (P1)
- Disk 1: Data (D3, D4), Parity (P2)
- Disk 2: Data (D5, D6), Parity (P3)

This rotation ensures no single disk is burdened with all parity operations, reducing bottlenecks.

Data Recovery: In case of a single disk failure, the missing data can be reconstructed by XOR-ing the remaining data blocks and parity information.

Evaluation

- **Reliability:** 1

RAID-5 allows recovery of at most 1 disk failure (because of the way parity works). If more than one disk fails, there is no way to recover the data. This is identical to RAID-4.

- **Capacity:** $(N-1) \times B$

Overall, space equivalent to one disk is utilized in storing the parity. Hence, $(N-1)$ disks are made available for data storage, each disk having B blocks.

Advantages

- Data can be reconstructed using parity bits.
- It makes the performance better.

Disadvantages

- Its technology is complex and extra space is required.
- If both discs get damaged, data will be lost forever.

RAID-6 (Block-Level Striping with two Parity Bits)

- RAID-6 is an advanced version of RAID-5 that provides enhanced fault tolerance by introducing double distributed parity. This allows RAID-6 to recover from the failure of up to two disks simultaneously, making it more reliable for critical systems with larger arrays. However, the added parity calculations can impact write performance.

RAID 6

Disk 1	Disk 2	Disk 3	Disk 4
A1	B1	P(B1)	P(B1)
A2	P(B2)	P(B2)	B2
P(B3)	P(B3)	A3	B3
P(B4)	A4	A4	P(B4)

How RAID-6 Works ?

Block-Level Striping: Data is divided into blocks and striped across all disks in the array.

Double Distributed Parity: Two sets of parity information are calculated for every block and distributed across all disks in the array in a rotating pattern.

Example:

- Disk 0: Data (D1, D2), Parity (P1)
- Disk 1: Data (D3, D4), Parity (P2)
- Disk 2: Data (D5, P1), Parity (P3)
- Disk 3: Parity (P2, P3), Data (D6)

Data Recovery: If one or two disks fail, the missing data can be reconstructed using the remaining data and parity information.

Evaluation

1. Reliability: RAID-6 can tolerate the simultaneous failure of two disks, providing greater fault tolerance than RAID-5.

2. Capacity: Usable Capacity = $(N-2) \times B$ where N is the total number of disks and B is the number of blocks per disk.

Advantages

- Very high data Accessibility.
- Fast read data transactions.

Disadvantages

- Due to double parity, it has slow write data transactions.
- Extra space is required.

Advantages of RAID

- **Data redundancy:** By keeping numerous copies of the data on many disks, RAID can shield data from disk failures.
- **Performance enhancement:** RAID can enhance performance by distributing data over several drives, enabling the simultaneous execution of several read/write operations.
- **Scalability:** RAID is scalable, therefore by adding more disks to the array, the storage capacity may be expanded.
- **Versatility:** RAID is applicable to a wide range of devices, such as workstations, servers, and personal PCs

Disadvantages of RAID

- **Cost:** RAID implementation can be costly, particularly for arrays with large capacities.
- **Complexity:** The setup and management of RAID might be challenging.
- **Decreased performance:** The parity calculations necessary for some RAID configurations, including RAID 5 and RAID 6, may result in a decrease in speed.
- **Single point of failure:** RAID is not a comprehensive backup solution while offering data redundancy. The array's whole contents could be lost if the RAID controller malfunctions.

Indexing and Hashing: Basic concept, Ordered Indices

Indexing and hashing are critical concepts in database management systems (DBMS) that enhance the efficiency of data retrieval operations.

Indexing

Indexing is a data structure technique to quickly locate and access the data in a database table. An index is similar to a book's index — it helps you find information without having to scan every item in the dataset.

Types of Indexing:

1. **Primary Index:** This is created on the primary key of a table. It ensures that the data can be retrieved efficiently based on the unique identifiers of rows.
2. **Secondary Index:** This is created on non-primary key columns to facilitate quick searches on those columns.
3. **Clustered Index:** This defines the order in which the data is physically stored in the table. Only one clustered index can exist per table.
4. **Non-Clustered Index:** This does not alter the physical order of the table and maintains a separate structure to point to the data.

Example of Indexing:

Consider a simple table called Students:

StudentID	Name	Age	Grade
1	Alice	20	A
2	Bob	21	B

3	Carol	22	A
4	Dave	23	C

If we create an index on the Name column, the DBMS builds a separate data structure that allows quick access to the records based on the student's name.

Hashing

Hashing is another method for fast data retrieval that converts an input (or 'key') into a fixed-size string of bytes. The output is typically used as an index in a hash table.

Characteristics of Hashing:

- **Fixed Size:** The hash table has a fixed size, which is beneficial for constant time complexity lookup.
- **Collision Handling:** Different keys may produce the same hash (a collision), and strategies like chaining or open addressing are used to resolve such collisions.

Example of Hashing:

Suppose we want to store a student's grade based on their StudentID. We could use a simplistic hash function like:

$\text{Hash}(\text{StudentID}) = \text{StudentID} \bmod \text{TableSize}$

Assuming our table size is 10, the hash values for the StudentID might look like:

StudentID	Hash Value
1	1
2	2
3	3
4	4

In this setup, you can quickly find a record by calculating its hash. If you search for StudentID = 2, you compute $2 \bmod 10 = 2$ and directly access the record.

Ordered Indices

Ordered indices maintain the sequence of the data in sorted order, which can be utilized for range queries effectively.

Example of Ordered Index:

Using the previous Students table, if we create an ordered index on the Age column, the index might look something like this:

Index (Age)	StudentID
20	1
21	2
22	3
23	4

This ordered index allows efficient searching for students within a specific age range, such as finding all students aged 21 to 22.

B+ tree Index Files: Structure of a B+- Tree

A B+ tree is a type of data structure commonly used in databases and filesystems for indexing. It's an extension of the B-tree, allowing for efficient data retrieval, insertion, and deletion. Here's a brief overview of B+ trees, followed by an example of how they work in the context of an Advanced Database Management System (ADBMS).

Characteristics of B+ Trees

1. **Balanced Structure:** All leaf nodes are at the same level, ensuring $O(\log n)$ time complexity for search, insert, and delete operations.

2. **Sorted Order:** Keys are stored in sorted order, which allows for efficient range queries.
3. **Leaf Nodes:** Leaf nodes contain pointers to actual data records, while internal nodes only store keys and pointers to child nodes.
4. **Order:** The order (m) of the tree defines the maximum number of children each node can have.

Example Scenario

Imagine we have a database table called Employees with the following attributes:

- EmployeeID (Primary Key)
- Name
- Department
- Salary

We want to create a B+ Tree index on the EmployeeID attribute for fast retrieval of employee records.

Properties of the B+ Tree

Order (m): Let's assume our B+ Tree is of order 3. This means each node can have a maximum of 2 keys and 3 children.

Leaf Nodes: Only leaf nodes store actual employee records.

Initial Setup

Suppose we start with an empty B+ Tree and insert the following EmployeeIDs: 10, 20, 5, 6, 15, 30, and 25.

Step-by-Step Insertion

Insert 10:

[10]

Insert 20:

[10, 20]

Insert 5:

[5, 10, 20]

Insert 6 (Node Overflows):

The node can only hold 2 keys, so we split it:

[10]

/ \

[5, 6] [20]

Insert 15:

Go to the right child:

[10]

/ \

[5, 6] [15, 20]

Insert 30:

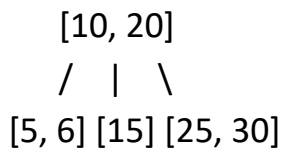
[10]

/ \

[5, 6] [15, 20, 30]

Insert 25 (Node Overflows):

Split the right child:



Leaf Nodes:

- [5, 6] contains EmployeeIDs 5 and 6.
- [15] contains EmployeeID 15.
- [25, 30] contains EmployeeIDs 25 and 30.

Searching for Employee Records

Example Search for EmployeeID 15:

1. Start at the root node [10, 20].
2. Since $15 > 10$ and $15 < 20$, traverse to the middle child.
3. Arrive at [15], which contains the EmployeeID. The corresponding record can be retrieved directly.

Advantages in ADBMS

1. **Efficient Searching:** The B+ Tree allows for logarithmic search time, making it efficient even with large datasets.
2. **Range Queries:** Since all leaf nodes are linked, retrieving a range of EmployeeIDs (e.g., between 10 and 30) can be done with minimal overhead.
3. **Balanced Structure:** Maintains balance automatically through splits, ensuring consistent performance.

Parameters	B+ Tree
Structure	Separate leaf nodes for data storage and internal nodes for indexing
Leaf Nodes	Leaf nodes form a linked list for efficient range-based queries
Order	Higher order (more keys)
Key Duplication	Typically allows key duplication in leaf nodes
Disk Access	Better disk access due to sequential reads in a linked list structure
Applications	Database systems, file systems, where range queries are common
Performance	Better performance for range queries and bulk data retrieval
Memory Usage	Requires more memory for internal nodes

Static Hashing and Dynamic Hashing

Static Hashing:

- In static hashing, a fixed amount of storage is allocated for the hash table at the start.
- The hash function maps keys to specific slots in the table, and collisions are handled using methods like chaining or open addressing.

- The main limitation is that if the number of entries exceeds the allocated space, performance degrades or the system may require rehashing.

Dynamic Hashing:

- Dynamic hashing allows the hash table to grow and shrink as needed.
- It uses a more sophisticated approach that involves dividing the hash table into buckets and managing them dynamically.
- This allows for better handling of variable-sized datasets and can improve performance by minimizing collisions and maintaining efficiency.

Example of Static Hashing

1. **Scenario:** Let's consider a simple database of student records with the following attributes: StudentID, Name, and Major.
2. **Data:**
 - StudentID: 101, Name: Alice, Major: Computer Science
 - StudentID: 102, Name: Bob, Major: Mathematics
 - StudentID: 103, Name: Charlie, Major: Physics
 - StudentID: 104, Name: Diana, Major: Computer Science
3. **Hash Function:** We will define a simple hash function that takes the StudentID modulo the size of the hash table. Let's say our hash table size is 5.

Hash Function: $h(\text{StudentID}) = \text{StudentID} \% 5$

Hash Table Creation:

- Using the hash function, we compute the indices for each student:

StudentID	Hash Index (h)
101	1
102	2
103	3
104	4

Handling Collisions:

- If multiple StudentIDs hash to the same index, we use chaining to store them in a linked list at that index. In this example, there are no collisions.

Hash Table Representation:

Index	Students
0	
1	101 → Alice
2	102 → Bob
3	103 → Charlie
4	104 → Diana

Query Example:

- To find the student with StudentID = 102, we compute:

$$h(102) = 102 \% 5 = 2$$

We look at index 2 in the hash table and retrieve Bob.

Dynamic Hashing in ADBMS

Definition: Dynamic hashing allows the hash table to grow and shrink as needed. It uses a more sophisticated structure to manage buckets and can handle variable-sized datasets efficiently.

Example of Dynamic Hashing

1. **Scenario:** Let's use the same student records but allow for dynamic resizing of the hash table.
2. **Initial Data:** Start with the same students as before.
3. **Initial Hash Function:**
 - Assume we start with a hash table of size 4.

Hash Function: $h(\text{StudentID}) = \text{StudentID} \% 4$

Initial Hash Table Creation:

- Calculate the indices:

StudentID	Hash Index (h)
101	1
102	2
103	3
104	0

Hash Table Representation:

Index	Students
0	104 → Diana
1	101 → Alice
2	102 → Bob
3	103 → Charlie

Inserting New Data:

Now, let's insert a new student: StudentID = 105, Name: Eve, Major: Mathematics.

The hash index would be calculated as

$$h(105) = 105 \% 4 = 1$$

This leads to a collision since index 1 already contains Alice.

Dynamic Resizing:

- To resolve this, the hash table needs to be resized. We double the size to 8 and rehash the existing records.

New Hash Function

New Hash Function: $h(\text{StudentID}) = \text{StudentID} \% 8$

1. Rehashing:

- Recalculate indices for all existing students:

StudentID	New Hash Index (h)
101	5
102	2
103	3
104	0

New Hash Table Representation:

Index	Students
0	104 → Diana
1	
2	102 → Bob
3	103 → Charlie
4	
5	101 → Alice
6	
7	

11. Insert New Student:

- Now, insert Eve (StudentID = 105):

$$h(105) = 105 \% 8 = 1$$

- This index is free, so Eve is added.

12. Final Hash Table Representation:

Index	Students
0	104 → Diana
1	105 → Eve
2	102 → Bob
3	103 → Charlie
4	
5	101 → Alice
6	
7	

Static Hashing is straightforward and efficient for fixed datasets but can struggle with collisions and resizing. **Dynamic Hashing**, on the other hand, provides flexibility and efficient handling of varying data sizes, making it suitable for applications with dynamic workloads.

Feature	Static Hashing	Dynamic Hashing
Table Size	Fixed	Dynamic
Hash Function	Static and predefined	Adaptable to table size
Performance	Can degrade with excess entries	Maintains performance with growth
Memory Usage	Fixed allocation, potential wastage	Efficient, grows/shrinks as needed
Complexity of Implementation	Simpler, easier to implement	More complex due to resizing logic
Collision Handling	Basic methods (chaining, open addressing)	Advanced, redistributes entries
Use Cases	Fixed datasets, small databases	Fluctuating datasets, scalable systems

Query Processing: Selection Operation

The selection operation in a database management system (DBMS) retrieves rows that satisfy a specific condition. This is often implemented using hashing for efficient data access.

Benefits of Selection Operation

- **Focused Analysis:** By selecting specific data, users can concentrate on information directly related to their needs. This targeted approach facilitates in-depth analysis and data-driven decision-making.
- **Efficiency:** Selection queries significantly improve efficiency by enabling users to retrieve only the required data, saving time and resources compared to processing entire datasets.
- **Accuracy:** Specifying selection criteria enhances the accuracy of results by ensuring retrieved data aligns with the user's specific requirements.

Selection Operation in Query Processing in DBMS

Picking out certain information from a database

Example :

```
SELECT column1, column2, ...
```

```
FROM table_name
```

Selection using File scans and Indices

Now, to select the particular column say CustomerName from the table we will type:

SELECT CustomerName FROM Customer;

Also, you can select multiple values like Customer Name, Age and Country:

SELECT CustomerName, Age FROM Customer;

by using * you can select all the attributes of the table:

Select * FROM Customer;

Selection Operation with Indexes

Conditions

Various conditions can be used with the help of 'WHERE' clause:

SELECT CustomerName, Age, CustomerID FROM Customer

WHERE Age>22;

Selection Operations with Comparisons

Implementing Complex Selection Operations -Picking out certain rows use-s many conditions. Conditions join with words like AND, OR, NOT.

SELECT *

FROM customer

WHERE (Country = 'Australia' OR LastName = 'Arpan') AND Age > 22;

Example of Selection Operation

Consider a simple database table Employees:

EmployeeID	Name	Department
1	Alice	HR
2	Bob	IT
3	Charlie	IT
4	Diana	Finance

Let's say we want to perform a selection operation to find all employees in the "IT" department.

Static Hashing Example

1. **Hash Function:** Assume we use a hash function that maps department names to specific slots. For simplicity, let's say:

- HR → 0
- IT → 1
- Finance → 2

2. **Hash Table:**

- The hash table is created with fixed slots based on the hash function.

Index	Employees
0	Alice
1	Bob, Charlie
2	Diana

3. **Selection Query:**

- Query: SELECT * FROM Employees WHERE Department = 'IT';
- The hash function maps 'IT' to index 1, retrieving Bob and Charlie.